COMP 345 TERM PROJECT PROPOSAL

Frozen LLM Embeddings Need Not Beat TF–IDF

for Cuisine Classification from Ingredient Lists

Author: Ruoyu Gu

Project type: Representations

23 September 2026 (Version 2)

1. Project type

Representations. We compare two representations of the same ingredient list: classical TF–IDF, and a frozen Gemini embedding. Both representations are classified by the same linear probe. We additionally test whether combining their prediction probabilities or their features improves classification.

2. Hypothesis

A frozen embedding probe will not exceed the TF–IDF probe's cuisine-classification macro-F1 by 0.02 or more at the 100% training setting when both use the same linear model class. We evaluate this hypothesis using the paired difference across the three outer test folds and its uncertainty interval, rather than a point estimate alone.

3. Project setup

Dataset. What's Cooking (Yummly), 20-way cuisine classification. Each labeled example has a recipe id, a cuisine label, and an unordered list of ingredient strings. We use the 39,774 labeled recipes. The official unlabeled Kaggle test file is not used, because it cannot be scored.

Data splits. Before modeling, normalize each ingredient list using the shared surface rules below and use its sorted normalized list as a split-group key. Freeze a shuffled, stratified group three-fold partition and its random seed: all recipes with the same normalized ingredient list stay in one fold. Each round holds out one fold for testing and uses the other two for development. Reserve 15% of the development data as a stratified group validation set; the rest forms that round's training pool. Internal stacking folds preserve the same groups. Duplicate ingredient-list groups with conflicting cuisine labels are retained and reported as an ambiguity in the dataset. All four methods share these assignments. Validation data select hyperparameters, and outer-test data are used only for evaluation. The three outer rounds replace repeating training three times on a single fixed split.

LLM. Google Gemini embedding API, model `gemini-embedding-001`. The model is frozen: we call the API once per recipe and save the vector to disk. Before any outer-test evaluation, fix the configuration to `task_type=CLASSIFICATION`, `output_dimensionality=768`, and an input string formed by joining the sorted normalized ingredient strings with newline characters. Lowercase ingredient strings, trim and collapse whitespace, and sort ingredients lexicographically. We do not select task type, output dimension, or serialization template using validation performance. Generation scripts and analysis scripts are separate. We do not fine-tune the LLM.

Baseline representation. TF–IDF over ingredients with the same shared surface normalization. We do not merge semantically related ingredients or remove brand names; for example, "garlic" and "garlic cloves" remain distinct strings. Select tokenization and n-gram settings on validation: word unigrams, word unigrams plus bigrams, or whole-ingredient tokens. Word n-grams stay within ingredient boundaries. Vocabulary and IDF statistics are fitted on the current training subset only, including separately within each internal stacking fold.

4. Inputs, outputs, model class, loss, optimizer, and what is learned

Input. A recipe x: a list of ingredient strings (no quantities, no cooking steps).

Output. A cuisine label $\hat{y}$ in a fixed set of 20 classes (for example Italian, Mexican, Indian, Chinese).

Model class. For each representation $z(x)$, a linear probe

$$f_\theta(z) = \mathrm{softmax}(Wz + b), \qquad W \in \mathbb{R}^{20 \times \dim(z)}.$$

The TF–IDF probe and the embedding probe belong to the same class (multinomial logistic regression). They are trained separately. Weights are not shared. We do not put a nonlinear head on only one arm. The fusion classifiers also use regularized multinomial logistic regression.

Loss. Multiclass cross-entropy between $f_\theta(z(x))$ and the gold cuisine, with ℓ_2 regularization on W chosen on validation.

Optimizer. All official model training and reported results use a PyTorch implementation of multinomial logistic regression optimized with Adam. The learning rate, ℓ_2 coefficient, batch size, maximum epochs, and early-stopping rule are selected using the validation set. Scikit-learn logistic regression may be used only as an optional convergence and implementation check; it is not mixed into the primary result tables. Tune separately at each training proportion, keeping validation examples outside the final training subset.

What is learned. Only new probe weights $\theta = (W, b)$ for each feature family, and the probability-fusion classifier's weights. The Gemini embedding model is frozen (no fine-tuned weights). TF–IDF is a fixed transform after the vocabulary and IDF are fit on train. We do not learn new prompts.

Pipeline.

- TF–IDF arm: $x \mapsto \phi_{\mathrm{tfidf}}(x) \in \mathbb{R}^V \mapsto$ linear probe $\mapsto \hat{y}$.
- LLM arm: $x \mapsto$ frozen Gemini embedding $e(x) \in \mathbb{R}^d \mapsto$ linear probe $\mapsto \hat{y}$.
- Probability fusion: $[p_{\mathrm{tfidf}}(x); p_{\mathrm{LLM}}(x)] \in \mathbb{R}^{40} \mapsto$ fusion classifier $\mapsto \hat{y}$.
- Feature fusion: $[\widehat{\phi}_{\mathrm{tfidf}}(x); \alpha\widehat{e}(x)] \mapsto$ linear probe $\mapsto \hat{y}$.

Probability fusion. Concatenate the two models' 20-class probability vectors in the same class order. Within each current training subset, use an additional stratified group three-fold split to generate out-of-fold (OOF) probabilities: train the base models on two internal folds and predict the remaining fold, then repeat. Train the fusion classifier on these OOF predictions and their labels, rather than on predictions from models fitted on the same examples. Refit the base models on the entire current training subset for validation and outer-test prediction. Tune the fusion classifier using validation data only. Internal stacking folds generate training inputs; outer folds evaluate the complete pipeline.

Feature fusion. Standalone TF–IDF uses row-wise L2 normalization (`norm=l2`), matching the standard TF–IDF vectorizer default. Feature fusion uses this same normalized TF–IDF block together with an L2-normalized embedding block scaled by α, then trains a new linear probe. Select α and regularization on validation because the two feature blocks have different scales and dimensions. Preserve TF–IDF sparsity to control memory usage. This tests whether joint features provide useful signals beyond either representation alone.

5. Model evaluation

Primary metric. Macro-F1: the unweighted mean of the 20 one-vs-rest F1 scores. Classes are imbalanced (Italian about 19.7%, Mexican about 16.2%, Brazilian about 1.2%), so accuracy would be dominated by majority cuisines. Use the same fixed set of 20 classes for every evaluation and set `zero_division=0`. A class is never removed from the macro average because its training support is zero. If a test split has zero support for a class, retain that class in the fixed label set, assign F1 = 0, and report the support explicitly. The 2% and 5% settings are low-data exploratory settings because rare cuisines may have only a few training examples.

Secondary metrics. Accuracy (the original Kaggle metric) and per-class F1 (for analysis, not for the pre-registered claim).

Baseline. TF–IDF plus the same linear probe. The LLM system is the same probe on frozen embeddings. This matches the course template: a model on classical features versus a model on frozen foundation-model features. Compare both fusion methods with each standalone baseline to assess complementarity, reporting their additional training cost.

Expected result relative to the baseline. At the 100% training setting, report the mean paired difference and a 95% paired bootstrap interval. The primary interpretation is based on the interval's relationship to 0.02: an interval entirely below 0.02 supports the claim that the embedding advantage is smaller than two percentage points; an interval crossing 0.02 is inconclusive; an interval entirely at or above 0.02 provides evidence against the claim. The three outer folds are evaluation folds with overlapping training sets, not independent repetitions. The interval is conditional on the fitted models and observed evaluation samples and does not capture all training or split uncertainty. Resample complete normalized ingredient-list groups within each fold when forming the paired interval.

Learning curves. Use 2%, 5%, 10%, 20%, 40%, 60%, 80%, and 100% of each round's training pool. Use nested, stratified group subsets shared by all four methods, with validation and test sets fixed within the round. Refit TF–IDF for each subset; never use the full pool's vocabulary for a smaller setting. Here 100% excludes validation and test data. Plot mean test macro-F1 and standard deviation across the three outer folds to assess whether embeddings help more in low-data settings. These curves measure training-sample efficiency with a fixed validation budget.

Variability and uncertainty. Report individual fold scores, their mean and sample standard deviation, and paired differences between methods. Standard deviations are not confidence intervals, and the outer folds have overlapping training sets. The paired bootstrap interval and its three-way interpretation are defined above; resample complete normalized ingredient-list groups within each fold. The interval is conditional on fitted models and does not capture all training uncertainty.

Four methods, eight training proportions, and three outer folds give 96 evaluation settings, in addition to tuning and internal stacking fits. Freeze the experimental choices before inspecting outer-test results, and retain seeds, split assignments, hyperparameters, and predictions for reproducibility.

6. Failure analysis

The final report will explain when, how, and why the system fails.

When does it fail? We will focus on rare labels (Brazilian, Russian, Jamaican, and Irish) and recipes with ingredients shared across cuisines. Compare these errors across low-data and full-training settings.

How does it fail? We will read confusion matrices and per-class F1, then inspect examples where only one model is correct, both fail, or fusion corrects or introduces an error. Include ingredient lists, labels, predictions, and relevant probabilities.

Why does a representation fail? Examine shared vocabulary, missing distinctive ingredients, and wording variants. Base explanations on metrics and concrete examples; UMAP is supplementary and cannot establish why a classifier failed. Any normalization experiment is selected on validation. A frozen Gemini embedding may contain pretrained knowledge about ingredients and cuisines, so a low-data advantage is interpreted as transfer from a pretrained representation, not proof that embeddings are inherently superior to word-frequency features.

AI use disclosure

We used Grok (xAI) to help turn our project ideas into the LaTeX source for the original PDF, and to search for datasets and background material. We used OpenAI Codex to help incorporate the agreed revisions and instructor feedback into Version 2. The author is responsible for verifying the experimental design, implementation, and scientific claims.